EXP NO: 2	SUPERVISED LEARNING MODELS – IMPLEMENT SUPPORT VECTOR MACHINES (SVM) FOR BINARY/MULTICLASS CLASSIFICATION AND APPLY RANDOM FOREST MODELS TO ENHANCE PREDICTIONS

AIM:

To implement Support Vector Machine (SVM) and Random Forest classifiers for multiclass classification on the Iris dataset, and to evaluate and compare their performance using accuracy and confusion matrices.

ALGORITHM:

1. Install and Import Required Libraries:
   - Install numpy, pandas, matplotlib, seaborn and scikit-learn using pip, then import them.
2. Load Dataset:
   - Load the Iris dataset (150 samples, 4 features, 3 classes) using load_iris().
3. Split Dataset:
   - Use train_test_split() with test_size=0.3 and random_state=42.
4. Feature Scaling:
   - Apply StandardScaler: fit on the training set and only transform the test set (prevents data leakage).
5. Train the SVM Model:
   - Create SVC(kernel='linear') and fit it on the training data.
6. Train the Random Forest Model:
   - Create RandomForestClassifier(n_estimators=100) and fit it on the training data.
7. Predict and Evaluate:
   - Predict on the test set; compute accuracy_score() and confusion_matrix() for both models.
8. Visualize Results:
   - Plot the confusion matrices of both models as heatmaps using seaborn.
9. Compare the performance of both models.

CODE:

```
# 1. Install necessary libraries (if not already installed)
# pip install numpy pandas matplotlib seaborn scikit-learn

# 2. Import libraries
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, confusion_matrix
import seaborn as sns
import matplotlib.pyplot as plt

# 3. Load dataset
iris = load_iris()
X = iris.data
y = iris.target

# 4. Train-test split (split BEFORE scaling to avoid data leakage)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# 5. Feature scaling (fit on training data only)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# ---------- Part A: Support Vector Machine ----------
svm_model = SVC(kernel='linear')   # You can also try 'rbf', 'poly'
svm_model.fit(X_train, y_train)
y_pred_svm = svm_model.predict(X_test)

print("SVM Accuracy:", accuracy_score(y_test, y_pred_svm))
print("SVM Confusion Matrix:\n", confusion_matrix(y_test, y_pred_svm))

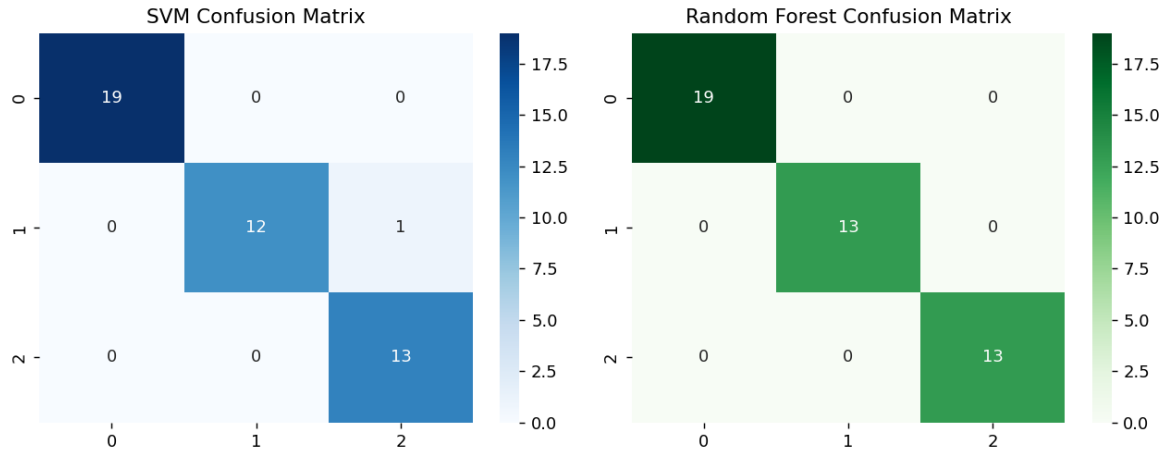
# ---------- Part B: Random Forest ----------
rf_model = RandomForestClassifier(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
y_pred_rf = rf_model.predict(X_test)

print("Random Forest Accuracy:", accuracy_score(y_test, y_pred_rf))
print("Random Forest Confusion Matrix:\n", confusion_matrix(y_test, y_pred_rf))

# ---------- Visualise confusion matrices ----------
plt.figure(figsize=(10, 4))
plt.subplot(1, 2, 1)
sns.heatmap(confusion_matrix(y_test, y_pred_svm), annot=True, cmap='Blues', fmt='d')
plt.title("SVM Confusion Matrix")
plt.subplot(1, 2, 2)
sns.heatmap(confusion_matrix(y_test, y_pred_rf), annot=True, cmap='Greens', fmt='d')
plt.title("Random Forest Confusion Matrix")
plt.tight_layout()
plt.show()
```

OUTPUT:

```
SVM Accuracy: 0.9777777777777777
SVM Confusion Matrix:
 [[19  0  0]
 [ 0 12  1]
 [ 0  0 13]]
Random Forest Accuracy: 1.0
Random Forest Confusion Matrix:
 [[19  0  0]
 [ 0 13  0]
 [ 0  0 13]]
```



RESULT:

The SVM (linear kernel) and Random Forest classifiers were successfully implemented on the Iris dataset. The SVM achieved an accuracy of 97.78% (44 of 45 test samples correct, with one Versicolor sample misclassified as Virginica), while the Random Forest achieved 100% accuracy on the test set. Thus, both models classify the Iris species effectively, with Random Forest giving slightly better predictions on this split.